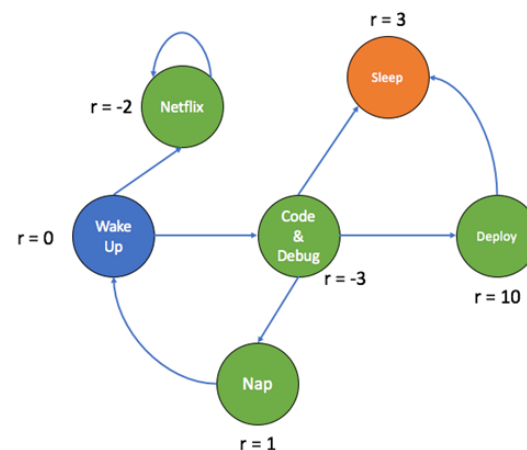


## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

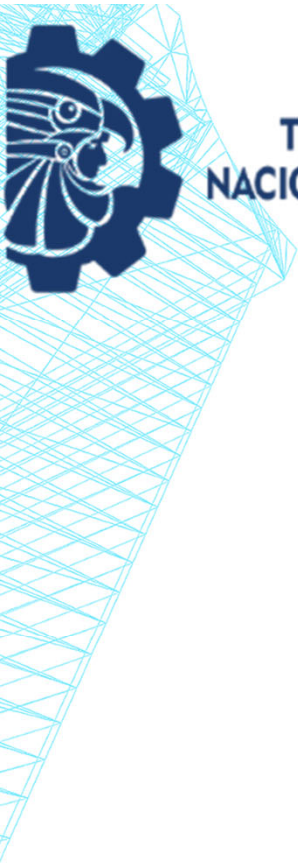
Markov decision processes or finite MDPs are a classical formalization of **sequential decision making**, where actions influence not just immediate rewards, but also subsequent situations, or states, and through those future rewards. Thus MDPs involve delayed reward and the need to tradeoff immediate and delayed reward.



## 3. Markov Decision Process

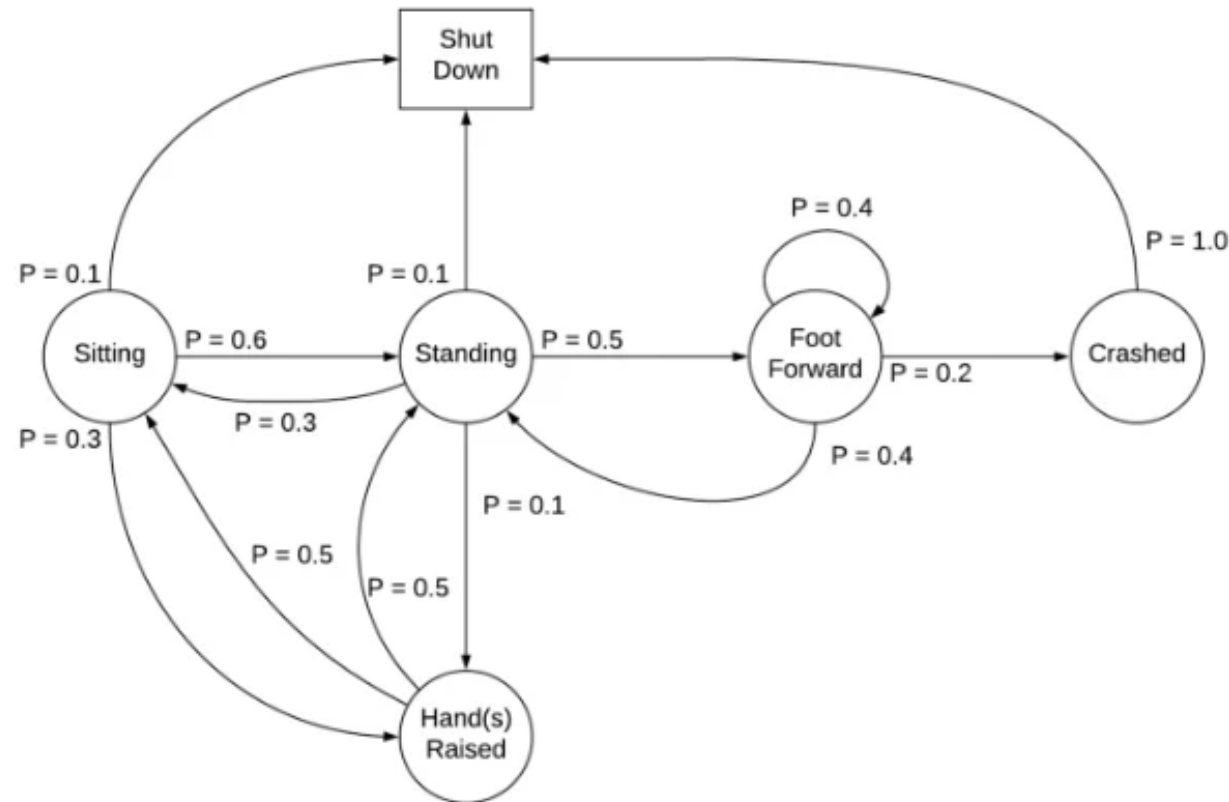
### 3.1 The Agent-Environment Interface

The k-Armed Bandit problem introduces many interesting concepts. However, it doesn't include many aspects of real-world problems. The agent is presented with **the same situation each time** and **the same action is always optimal**. In many problems, different situations call for different responses. The actions we choose now affect the amount of reward we can get into the future. The Markov Decision Process formalism captures these two aspects of real-world problems.



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface





## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

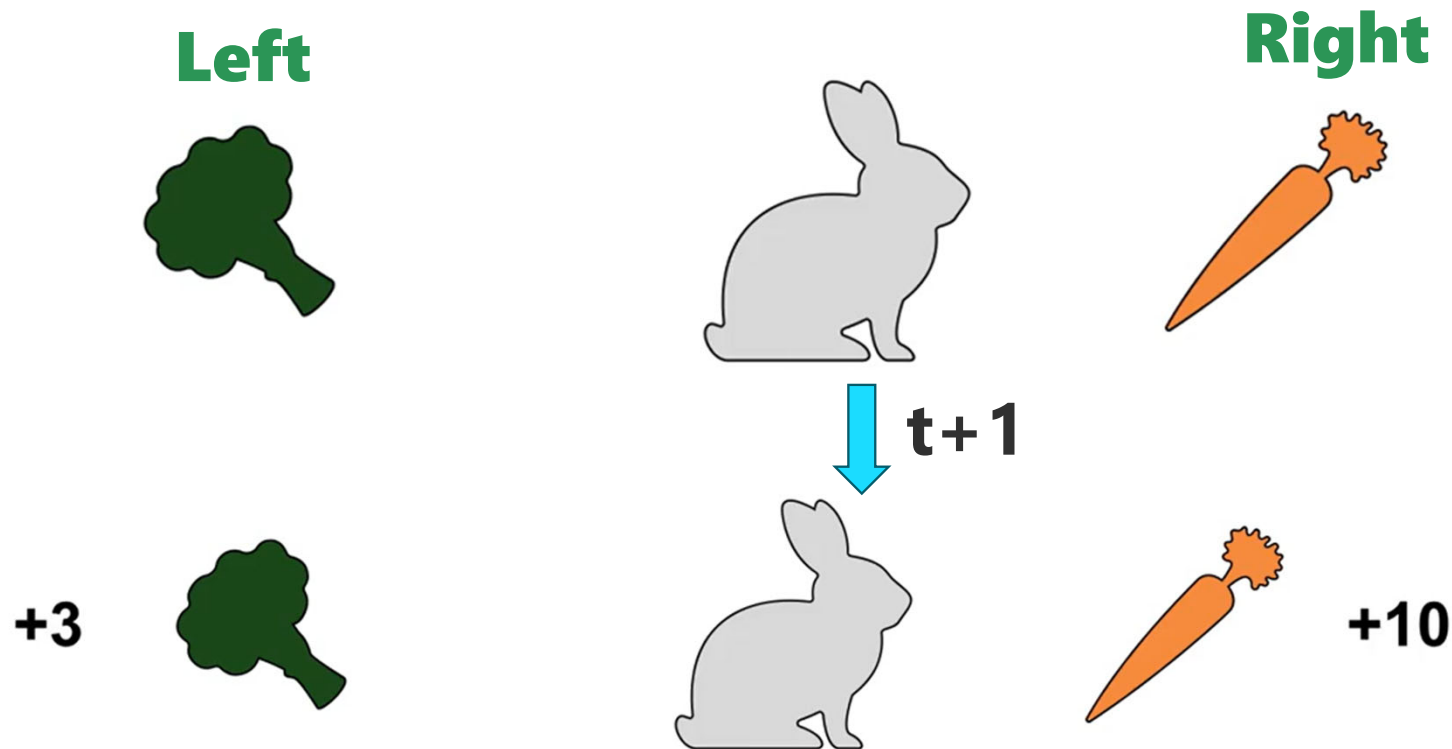
In many problems, different situations call for different responses. The actions we choose now affect the amount of reward we can get into the future. The Markov Decision Process formalism captures these two aspects of real-world problems.

## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example. Situation (1)

*"Different situations call different actions"*





TECNOLÓGICO  
NACIONAL DE MÉXICO

EDUCACIÓN

SECRETARÍA DE EDUCACIÓN PÚBLICA



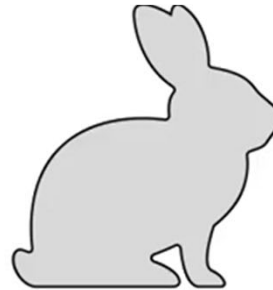
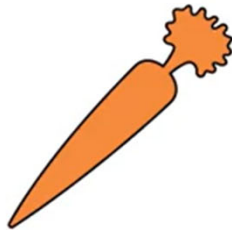
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example. Situation (2)

*"Different situations call different actions"*

**Left**



**Right**



**¿What would happen?**





TECNOLÓGICO  
NACIONAL DE MÉXICO

EDUCACIÓN

SECRETARÍA DE EDUCACIÓN PÚBLICA



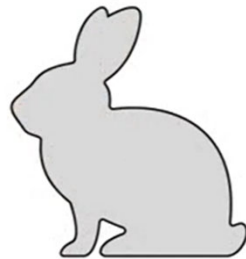
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example. Situation (2)

*"Different situations call different actions"*

**Left**



**Right**



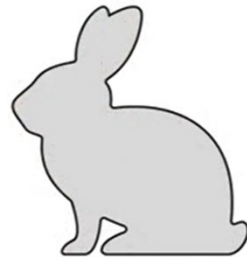


## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

The k-Armed Bandit problem does not account for the fact that different situations call for different actions.

Left



Right





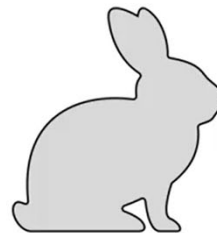
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

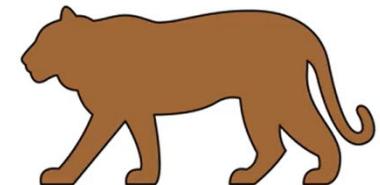
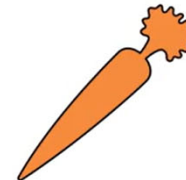
#### Example

The rabbit example. Situation (3)

Left



Right



## 3. Markov Decision Process

### 3.1 The Agent–Environment Interface

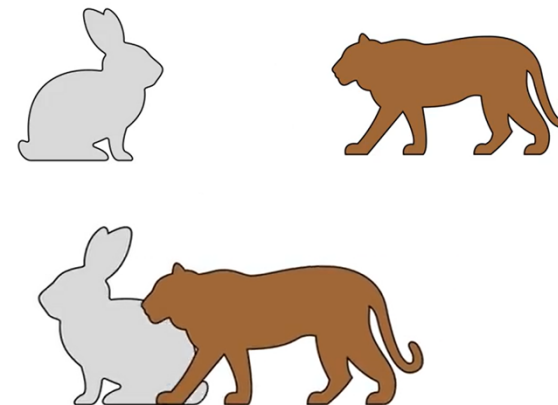
#### Example

The rabbit example. Situation (3)

Left



Right



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### Example

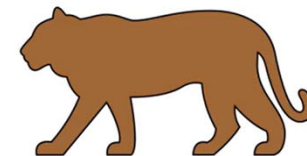
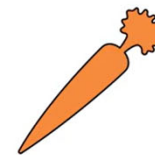
#### The rabbit example. Situation (3)

If we account for the long-term impact of our actions, the rabbit should go left and settle for broccoli to give itself a better chance to escape. **A bandit rabbit would only be concerned about immediate reward** and so it would go for the carrot. But a better decision can be made by considering the **long-term impact** of our decision.

Left



Right

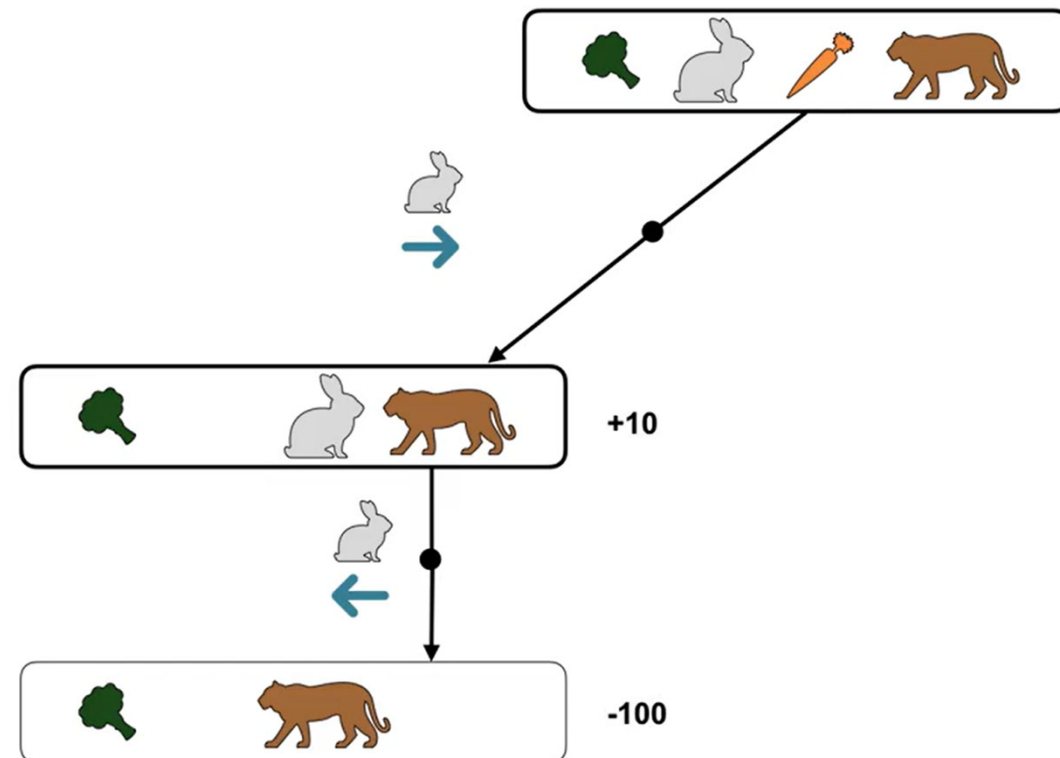


## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example.

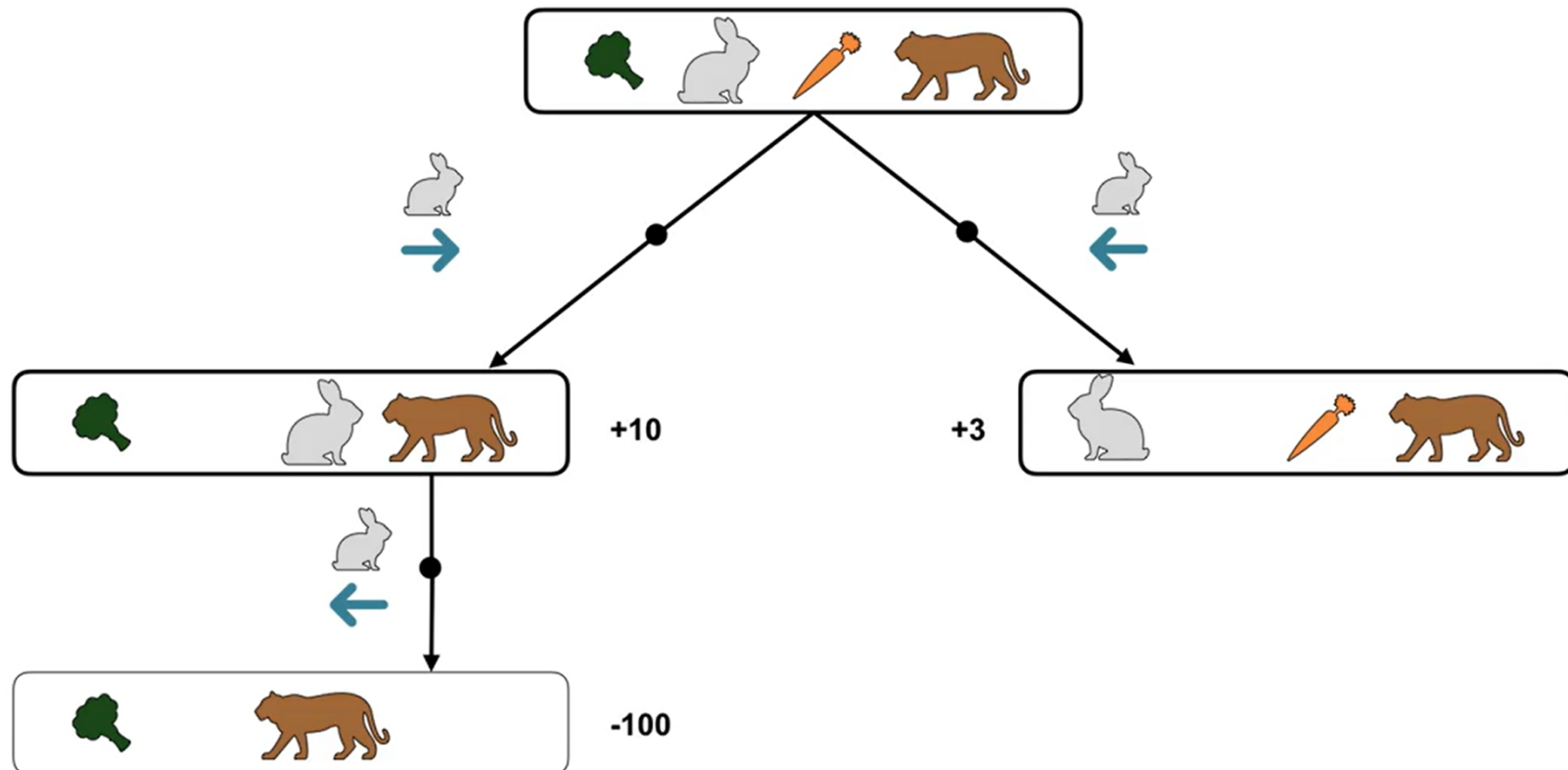
Look at how the situation changes as the rabbit takes actions. We will call these situations **states**. In each state the rabbit selects an action. For instance, the rabbit can choose to move right. Based on this action the world changes into a new state and produces a reward. In this case, the rabbit eats the carrot and receives a reward of plus 10. However, the rabbit is now next to the tiger. Let's say the rabbit chooses the left action. The world changes into a new state or the tiger eats the rabbit and the rabbit receives a reward of minus 100.



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

The rabbit example.





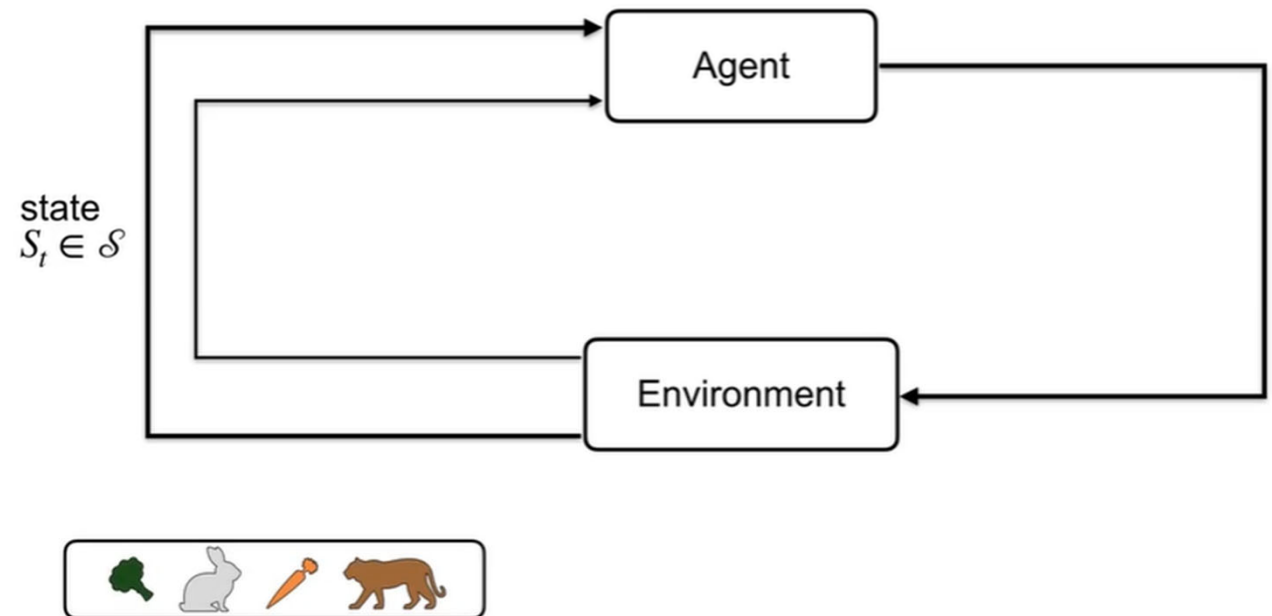
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example.

#### State ( $S_t$ )

The interaction can be mathematical formalized. In this framework, the **agent** and **environment** interact at **discrete time steps**. At each time, the agent receives a state  **$S_t$**  from the environment from a set of possible states, script  $S$ .





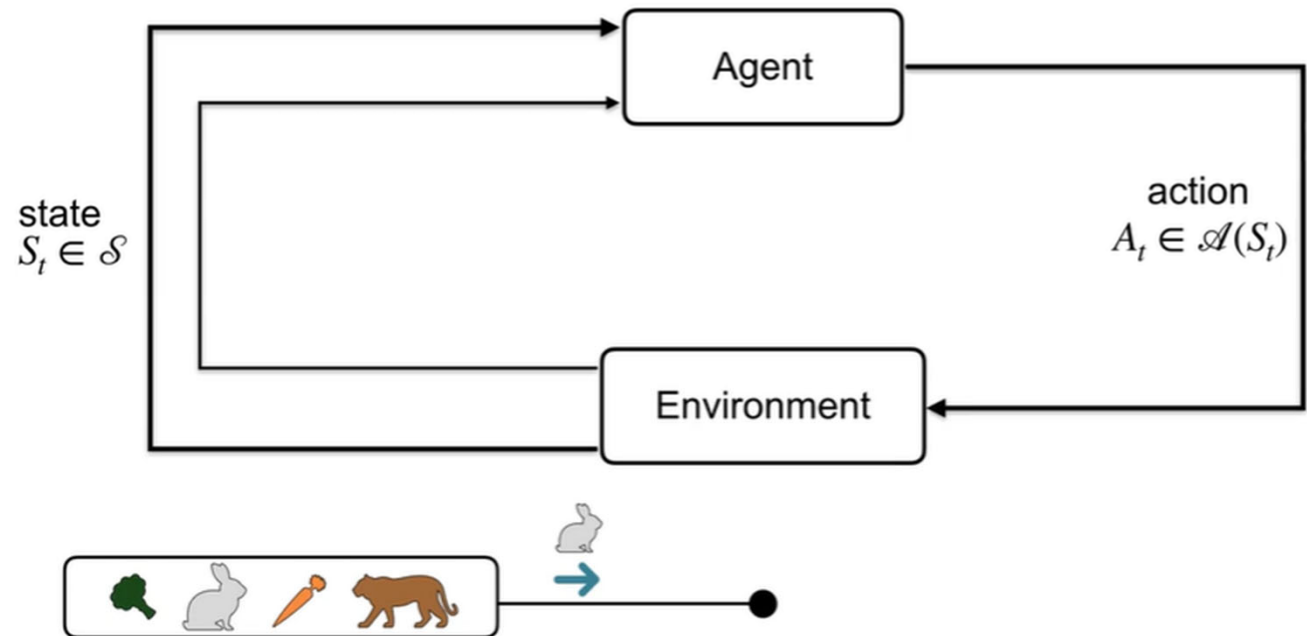
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example.

#### Action ( $A_t$ )

Based on this state the agent selects an action  **$A_t$**  from a set of possible actions. Script A of  **$S_t$**  is the set of valid actions in state  $S_t$ .





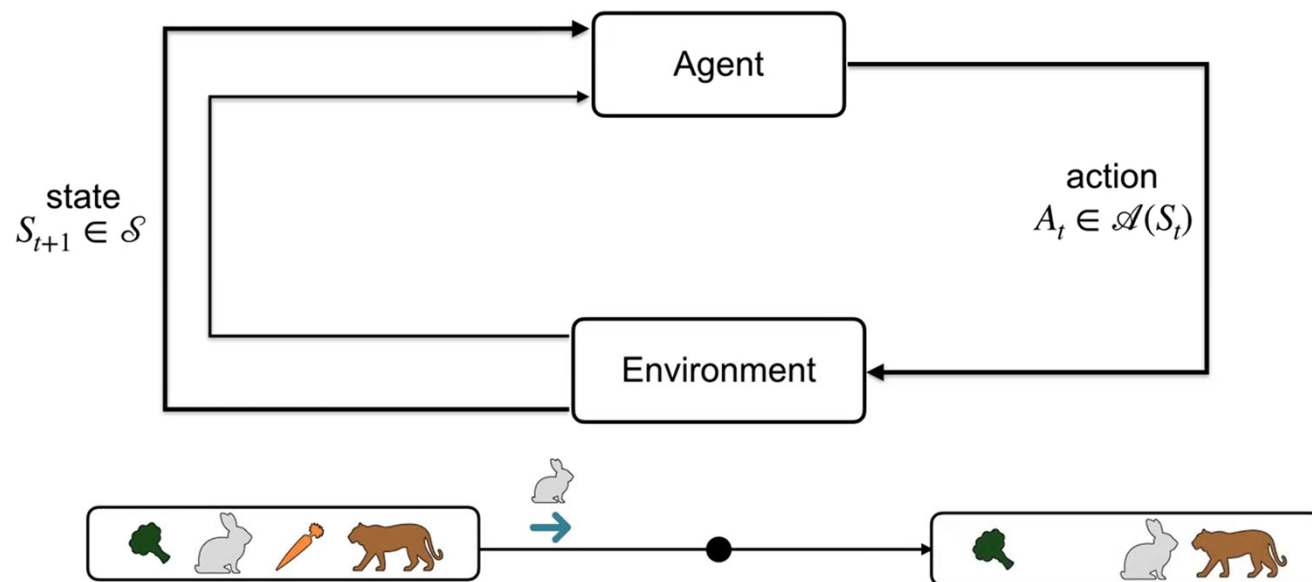
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example.

#### State ( $S_{t+1}$ )

Moving right is an example of an action. One time step later based in part on the agent's action, the agent finds itself in a new state  $S_{t+1}$ . For example, this state where the rabbit is next to the tiger.



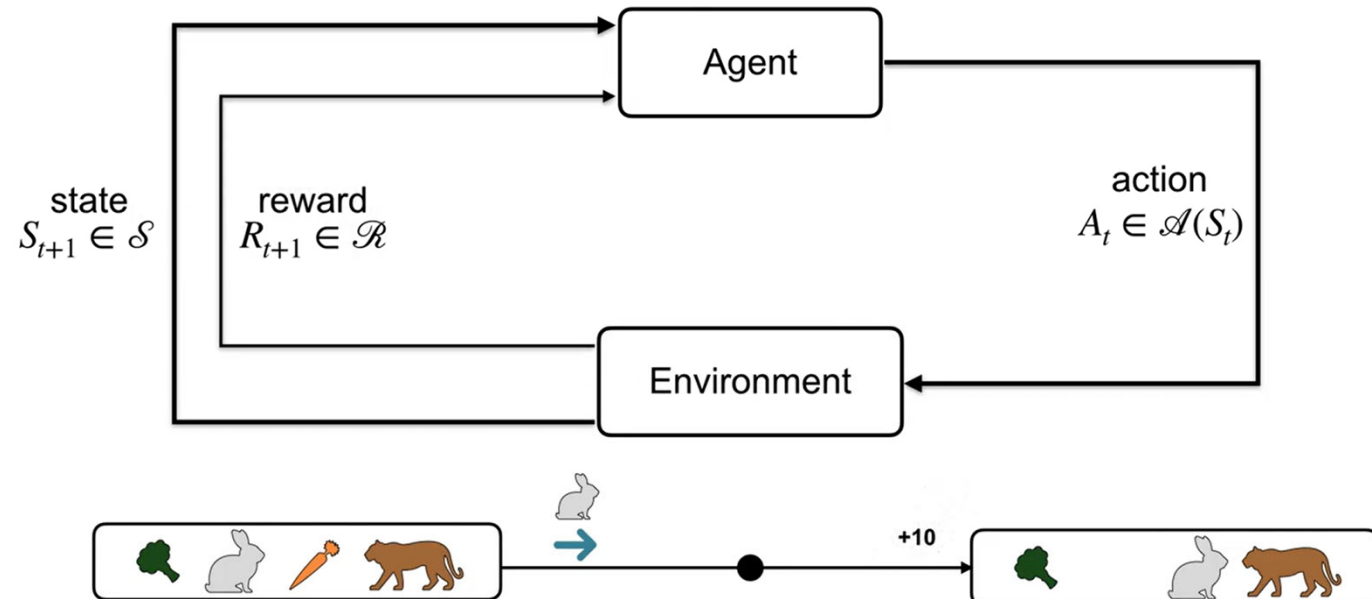
## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

#### The rabbit example.

#### Reward ( $R_{t+1}$ )

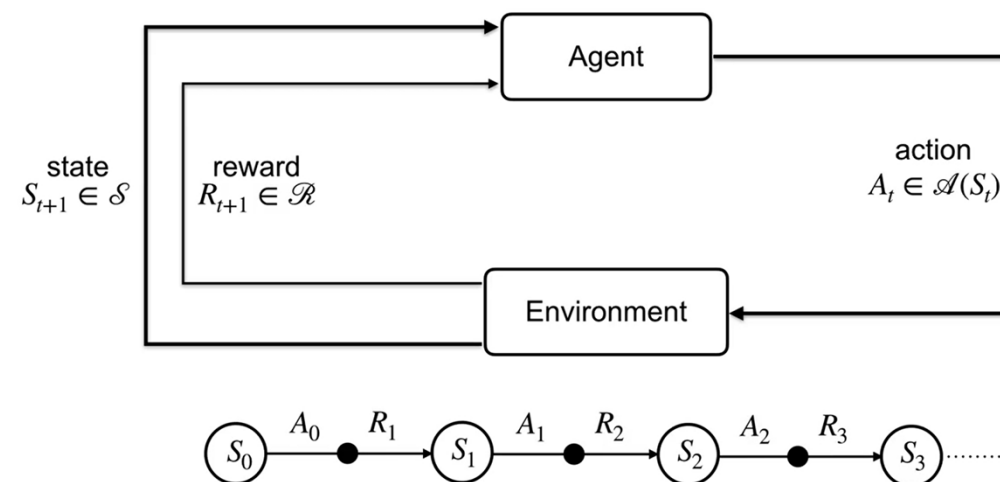
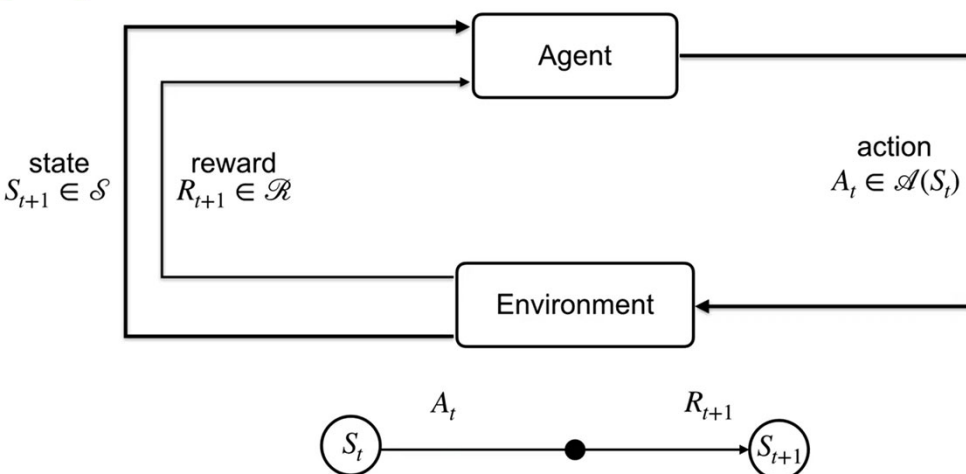
The environment also provides a scalar reward  $R_{t+1}$  drawn from a set of possible rewards, script  $\mathcal{R}$ . In this case, the reward is plus 10 for eating the carrot.



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

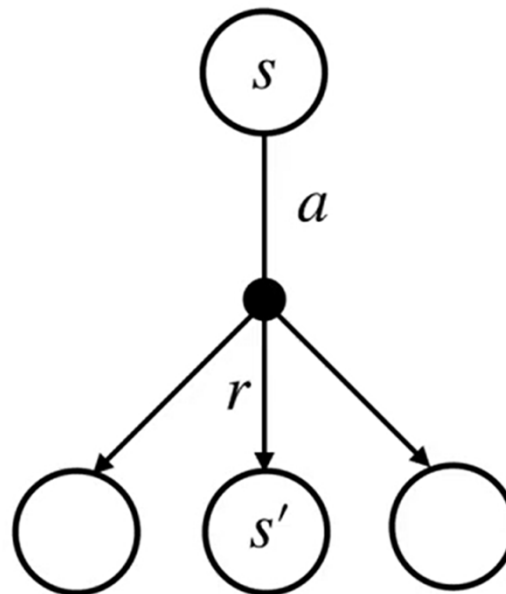
This diagram summarizes the agent environment interaction in the MDP framework. The agent environment interaction generates a **trajectory of experience** consisting of **states**, **actions**, and **rewards**. Actions influence immediate rewards as well as future states and through those, future rewards.



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

As in bandits, the outcomes are stochastic and so we use the language of probabilities. When the agent takes an action in a state, there are many possible next states and rewards.

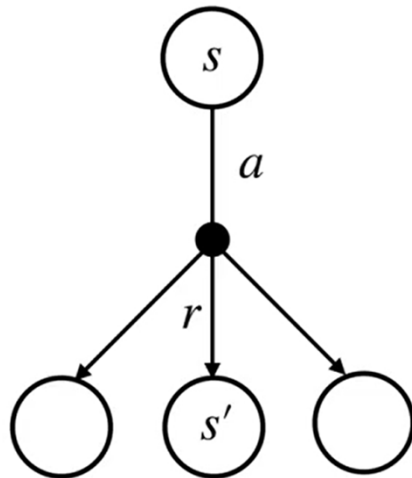




## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

The transition dynamics function  $P$ , formalizes this notion. Given a state  $\mathbf{s}$  and action  $\mathbf{a}$ ,  $\mathbf{p}$  tells us the joint probability of next state  $\mathbf{s}$  prime and reward are.



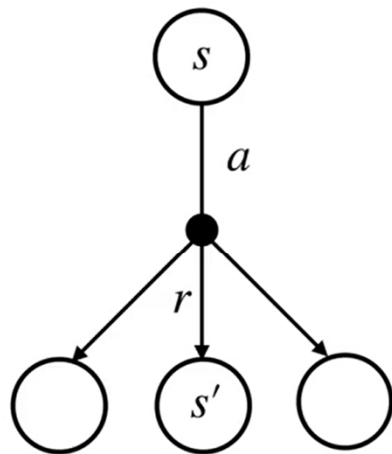
$$p(s', r | s, a)$$



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

Since  $\mathbf{p}$  is a probability distribution, it must be non-negative and its sum over all possible next states and rewards must equal one. Note that future state and reward only depends on the current state and action. This is called the **Markov property**. It means that ***the present state is sufficient and remembering earlier states would not improve predictions about the future.***



$$p(s', r | s, a)$$

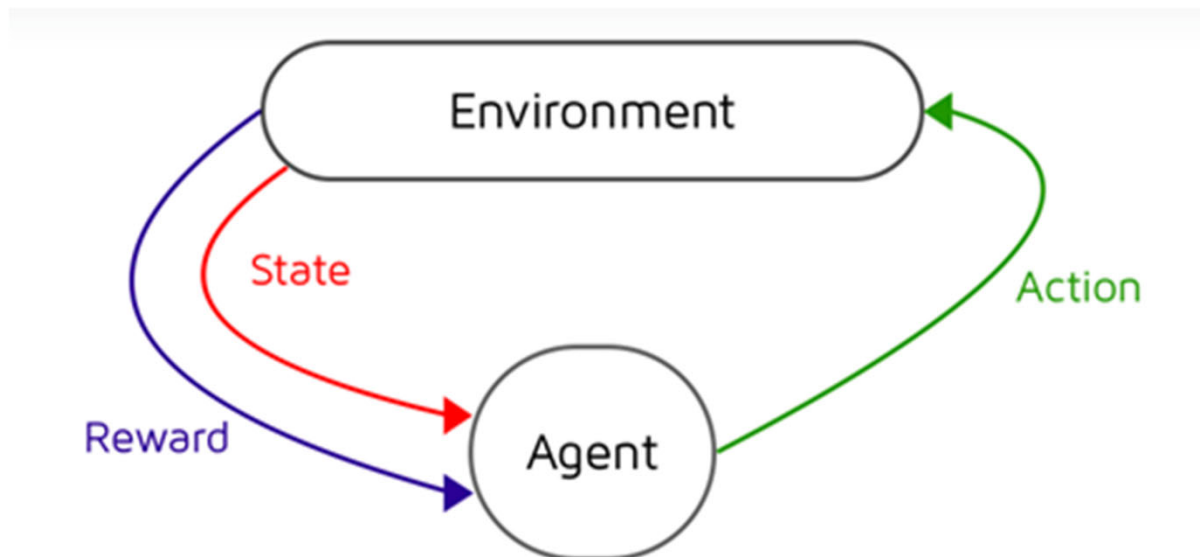
$$p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

- MDPs provide a general framework for sequential decision making.
- The dynamics of an MDP are defined by a probability distribution.



## 3. Markov Decision Process

### 3.1 The Agent-Environment Interface

Let's play: Win or die satisfied. The Rabbit's dilemma!

**Python Example**  
mdp\_01.py



## 3. Markov Decision Process

### 3.2 Markov Property

#### A rigorously theoretical study

In a *finite* MDP, the sets of states, actions, and rewards ( $\mathcal{S}$ ,  $\mathcal{A}$ , and  $\mathcal{R}$ ) all have a finite number of elements. In this case, the random variables  $R_t$  and  $S_t$  have well defined discrete probability distributions dependent only on the preceding state and action. That is, for particular values of these random variables,  $s' \in \mathcal{S}$  and  $r \in \mathcal{R}$ , there is a probability of those values occurring at time  $t$ , given particular values of the preceding state and action:

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}, \quad (3.2)$$

for all  $s', s \in \mathcal{S}$ ,  $r \in \mathcal{R}$ , and  $a \in \mathcal{A}(s)$ . The function  $p$  defines the *dynamics* of the MDP.



## 3. Markov Decision Process

### 3.2 Markov Property

#### A rigorously theoretical study

The dot over the equals sign in the equation reminds us that it is a definition (in this case of the function  $p$ ) rather than a fact that follows from previous definitions. The dynamics function  $p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$  is an ordinary deterministic function of four arguments. The ‘|’ in the middle of it comes from the notation for conditional probability, but here it just reminds us that  $p$  specifies a probability distribution for each choice of  $s$  and  $a$ , that is, that

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s). \quad (3.3)$$

## 3. Markov Decision Process

### 3.2 Markov Property

#### A rigorously theoretical study

In a *Markov* decision process, the probabilities given by  $p$  completely characterize the environment's dynamics. That is, the probability of each possible value for  $S_t$  and  $R_t$  depends only on the immediately preceding state and action,  $S_{t-1}$  and  $A_{t-1}$ , and, given them, not at all on earlier states and actions. This is best viewed a restriction not on the decision process, but on the *state*. The state must include information about all aspects of the past agent–environment interaction that make a difference for the future. If it does, then the state is said to have the *Markov property*.



## 3. Markov Decision Process

### 3.2 Markov Property

#### DETERMINISM VERSUS STOCHASTIC



random

stochastic



TECNOLÓGICO  
NACIONAL DE MÉXICO

EDUCACIÓN

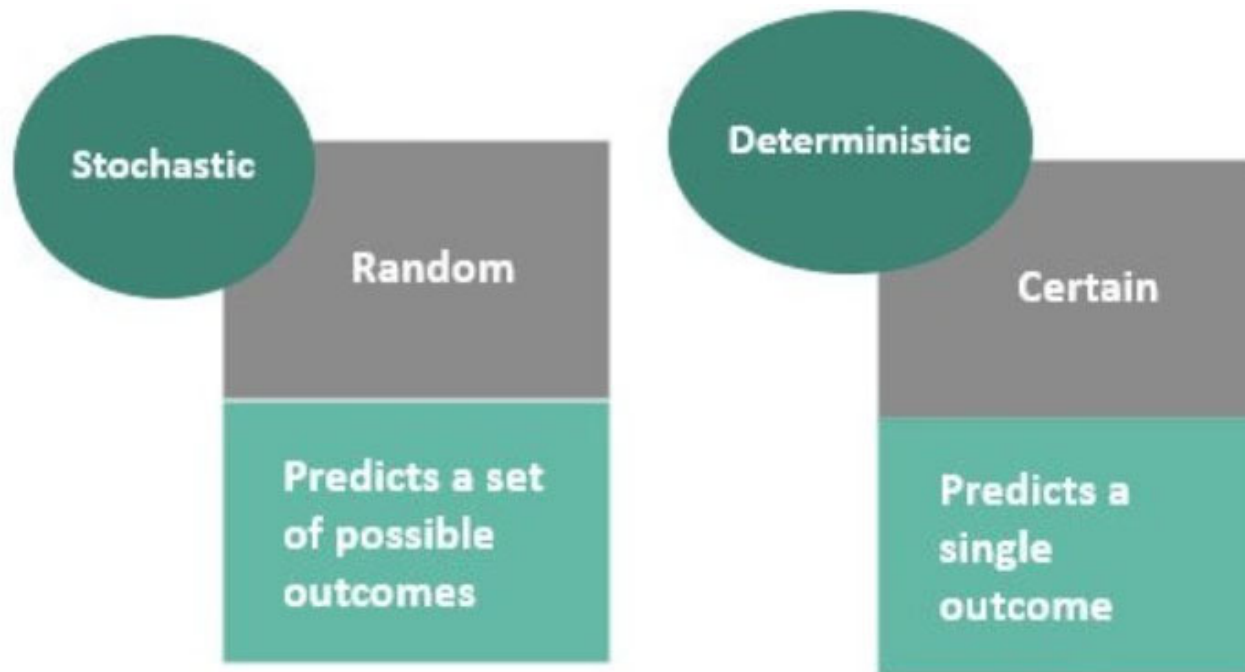
SECRETARÍA DE EDUCACIÓN PÚBLICA



## 3. Markov Decision Process

### 3.2 Markov Property

#### DETERMINISM VS STOCHASTIC



## 3. Markov Decision Process

### 3.2 Markov Property

#### Students Lecture – OPTIONAL – pp. 47-53

From the four-argument dynamics function,  $p$ , one can compute anything else one might want to know about the environment, such as the *state-transition probabilities* (which we denote, with a slight abuse of notation, as a three-argument function  $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ ),

$$p(s' | s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a). \quad (3.4)$$

## 3. Markov Decision Process

### 3.2 Markov Property

Students Lecture – pp. 47-53

We can also compute the expected rewards for state–action pairs as a two-argument function  $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ :

$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1}=s, A_{t-1}=a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a), \quad (3.5)$$

## 3. Markov Decision Process

### 3.2 Markov Property

#### Students Lecture – pp. 47-53

and the expected rewards for state–action–next-state triples as a three-argument function  $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ ,

$$r(s, a, s') \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}. \quad (3.6)$$



## 3. Markov Decision Process

### 3.2 Markov Property

#### Findings pp. 47-53

The MDP framework is a considerable abstraction of the problem of goal-directed learning from interaction. It proposes that whatever the details of the sensory, memory, and control apparatus, and whatever objective one is trying to achieve, any problem of learning goal-directed behavior can be reduced to three signals passing back and forth between an agent and its environment: one signal to represent the choices made by the agent (the actions), one signal to represent the basis on which the choices are made (the states), and one signal to define the agent's goal (the rewards). This framework may not be sufficient to represent all decision-learning problems usefully, but it has proved to be widely useful and applicable.

## 3. Markov Decision Process

### 3.2 Markov Property

#### Example

This robot works in a laboratory where it collects aluminum cans using mechanical grippers (claws). However, its grippers are old and worn out, so they sometimes fail while trying to **grab** the cans.

- It is approximately 70% likely when it is **searching** that the robot **found a can** and it would receive a reward of +10 points.
- But because its grippers are unreliable, sometimes the robot drops a can. When that happens, it receives a penalty of -1 point.

The robot's goal is to collect as many cans as possible and maximize its total score by successfully grabbing and keeping the cans, while minimizing the number of times it drops them.



## 3. Markov Decision Process

### 3.2 Markov Property

#### Students Exercise

Try to draw the MDP transition graph for the dynamics of the can robot. When you finish, draw it in the whiteboard.





## 3. Markov Decision Process

### 3.2 Markov Property

#### Students Exercise

Fill the table

| States ( $s$ ) | Action ( $a$ ) | $s'$ | $p(s', r   s, a)$ | $r(s, a, s')$ |
|----------------|----------------|------|-------------------|---------------|
|                |                |      |                   |               |
|                |                |      |                   |               |



## 3. Markov Decision Process

### 3.2 Markov Property

**Example – Can Robot v2 - PYTHON**





## 3. Markov Decision Process

### 3.2 Markov Property

#### DETERMINISM VERSUS STHOCASTIC

**Stochastic** (The Event): This refers to the uncertain outcome. It's the "what happened in the end." You can't predict it with 100% certainty.

```
def simulate_trial(state, action):  
    dynamics = []  
    probabilities = []  
  
    for (s, a, s_next, r), prob in can_robot_dynamics.items():  
        if s == state and a == action:  
            dynamics.append((s_next, r))  
            probabilities.append(prob)  
  
    return random.choices(dynamics, weights=probabilities, k=1)[0]
```



## 3. Markov Decision Process

### 3.2 Markov Property

#### DETERMINISM VERSUS STHOCASTIC

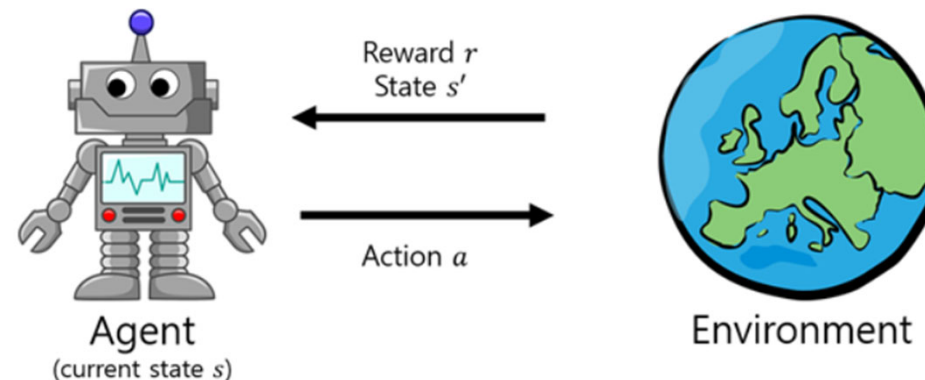
**Determinism** (The Rule): This refers to the probabilities that govern randomness. It is the world's "physics manual".

```
can_robot_dynamics = {  
    ("search", "grab", "can_found", 10): 0.7,  
    ("search", "grab", "search", -1): 0.3  
}
```

## 3. Markov Decision Process

### 3.3 Goals and Rewards

- In RL, the purpose or goal of the agent is formalized in terms of a special signal, called the **reward**, passing from the environment to the agent. At each time step, the reward is a simple number,  $R_t \in \mathbb{R}$ .
- Informally, the agent's goal is to maximize the total amount of reward it receives. This means maximizing not immediate reward, but cumulative reward in the long run.



## 3. Markov Decision Process

### 3.3 Goals and Rewards

#### Reward Hypothesis:

*"That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward)."*

#### Students

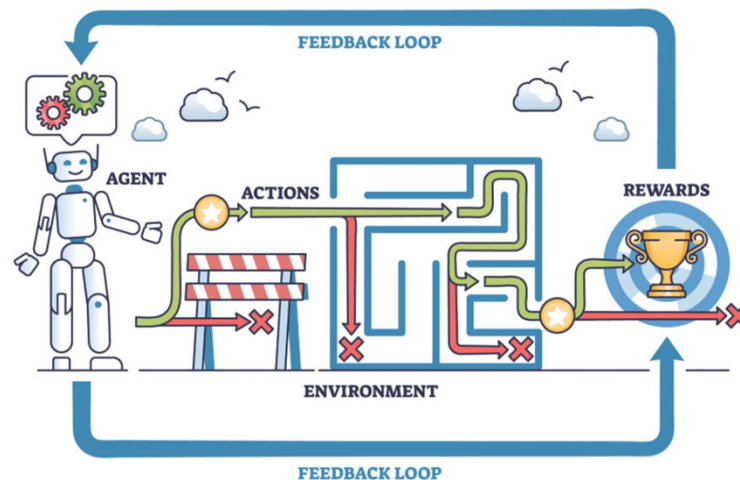
What does that mean ?

## 3. Markov Decision Process

### 3.3 Goals and Rewards

### Reward Hypothesis

- The hypothesis posits that any goal or objective, no matter how complex, can be reduced to a single number (a scalar) that the agent seeks to maximize over time.
- From learning to play chess to a robot learning to walk or even cook, everything can be “translated” into this language of rewards.



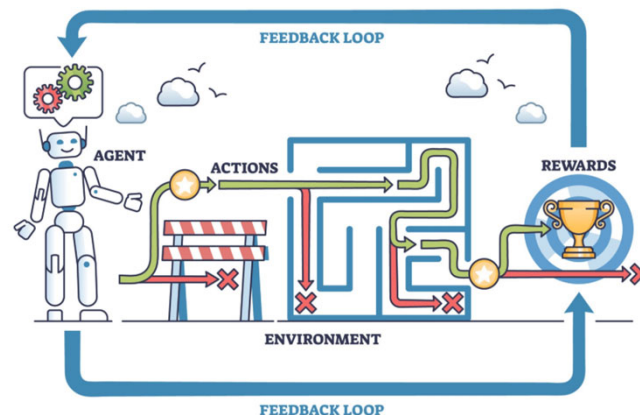


## 3. Markov Decision Process

### 3.3 Goals and Rewards

#### Is this important?

- It depends in your application.
- No matter how varied the objectives may be (speed, security, energy savings), they must ultimately be summed up into a single number. If you have multiple objectives, you must decide how much one is worth in terms of the other.
- The agent cares not only about what it receives “right now,” but also about the sum of everything it will receive in the future (the math form comes later).
- Since the environment is often uncertain (we don’t know exactly what will happen next), the goal is to maximize the average of what we expect to receive.

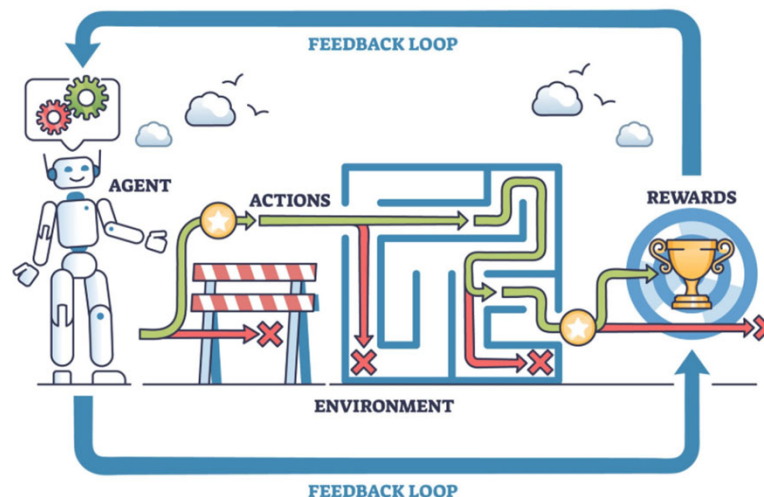


## 3. Markov Decision Process

### 3.3 Goals and Rewards

#### Facts

- If we accept this hypothesis, the problem of “being intelligent” or “achieving a goal” becomes a mathematical optimization problem.
- If the agent fails, it is not because it “is not intelligent,” but because the reward function is poorly designed or the agent has not explored enough to figure out how to maximize it.
- **Example:** In a video game, the goal isn’t “to survive”; the goal is to maximize points. If we set it so that surviving earns +1 point every second, the agent will understand that surviving is the path to winning.



## 3. Markov Decision Process

### 3.3 Goals and Rewards

#### More Examples

Although formulating goals in terms of reward signals might at first appear limiting, in practice it has proved to be flexible and widely applicable.

- To make a robot learn to walk, researchers have provided reward on each time step proportional to the robot's forward motion.
- In making a robot learn how to escape from a maze, the reward is often  $-1$  for every time step that passes prior to escape; this encourages the agent to escape as quickly as possible.
- To make a robot learn to find and collect empty soda cans for recycling, one might give it a reward of zero most of the time, and then a reward of  $+1$  for each can collected. One might also want to give the robot negative rewards when it bumps into things or when somebody yells at it.
- For an agent to learn to play checkers or chess, the natural rewards are  $+1$  for winning,  $-1$  for losing, and  $0$  for drawing and for all nonterminal positions.

## 3. Markov Decision Process

### 3.3 Goals and Rewards

#### More Examples

The agent always learns to maximize its reward. If we want it to do something for us, we must provide rewards to it in such a way that in maximizing them the agent will also achieve our goals. It is thus critical that the rewards we set up truly indicate what we want accomplished.

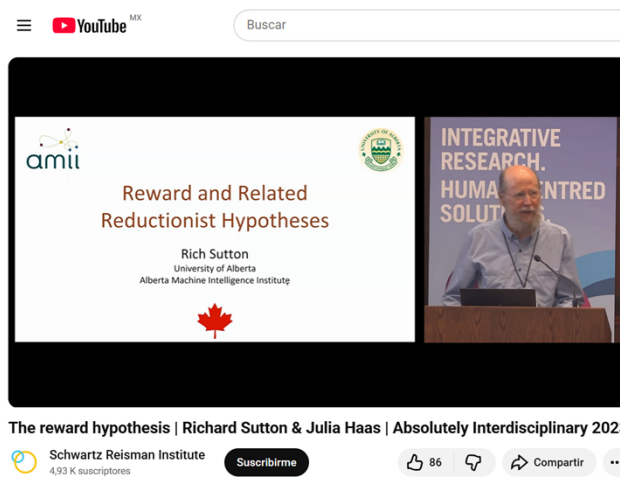
- For example, a chess-playing agent should be rewarded only for actually winning, not for achieving subgoals such as taking its opponent's pieces or gaining control of the center of the board.
- If achieving these sorts of subgoals were rewarded, then the agent might find a way to achieve them without achieving the real goal.
- For example, it might find a way to take the opponent's pieces even at the cost of losing the game. The reward signal is your way of communicating to the robot what you want it to achieve, not how you want it achieved.

## 3. Markov Decision Process

### 3.3 Goals and Rewards

#### Lectures and video

- <http://incompleteideas.net/rlai.cs.ualberta.ca/RLAI/rewardhypothesis.html>
- <https://arxiv.org/pdf/2212.10420>
- <https://www.youtube.com/watch?v=SLqbhllshr0&t=554s>





## 3. Markov Decision Process

### 3.3 Goals and Rewards

Richard S. Sutton

- [https://scholar-google-com.translate.goog/citations?user=6m4wv6gAAAAJ&hl=en&x\\_tr\\_sl=en&x\\_tr\\_tl=es&x\\_tr\\_hl=es&x\\_tr\\_p to=tc](https://scholar-google-com.translate.goog/citations?user=6m4wv6gAAAAJ&hl=en&x_tr_sl=en&x_tr_tl=es&x_tr_hl=es&x_tr_p to=tc)
- <https://www.scopus.com/authid/detail.uri?authorId=36172174400>

Richard S. Sutton

Informático



Fuente: The Toronto Penny Office & Wealth Mana...



Richard S. Sutton

Keen, Amii, and University of Alberta  
Verified email at richsutton.com - Homepage  
artificial intelligence reinforcement learning machine learning cognitive science  
computer science

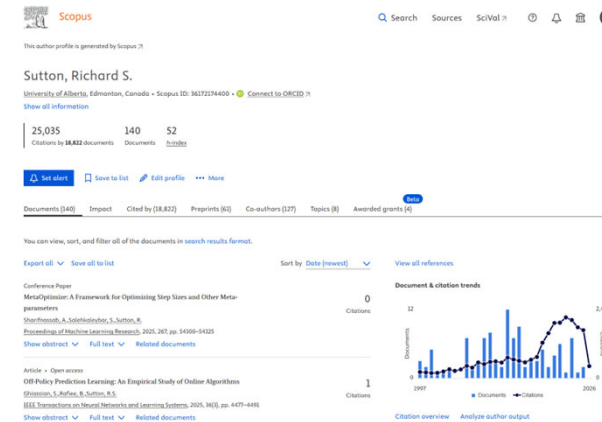
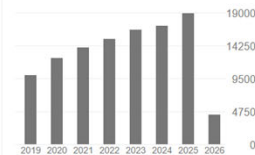
FOLLOW

GET MY OWN PROFILE

| TITLE                                                                          | CITED BY | YEAR |
|--------------------------------------------------------------------------------|----------|------|
| Reinforcement learning: An introduction                                        | 94058    | 1998 |
| Policy gradient methods for reinforcement learning with function approximation | 10868    | 1999 |
| Learning to predict by the methods of temporal differences                     | 9072     | 1988 |
| Reinforcement learning: An Introduction, 1st edition                           | 6424 *   | 1998 |

Cited by VIEW ALL

|           | All    | Since 2021 |
|-----------|--------|------------|
| Citations | 182335 | 86346      |
| h-index   | 100    | 65         |
| i10-index | 251    | 178        |





TECNOLÓGICO  
NACIONAL DE MÉXICO

## 3. Markov Decision Process

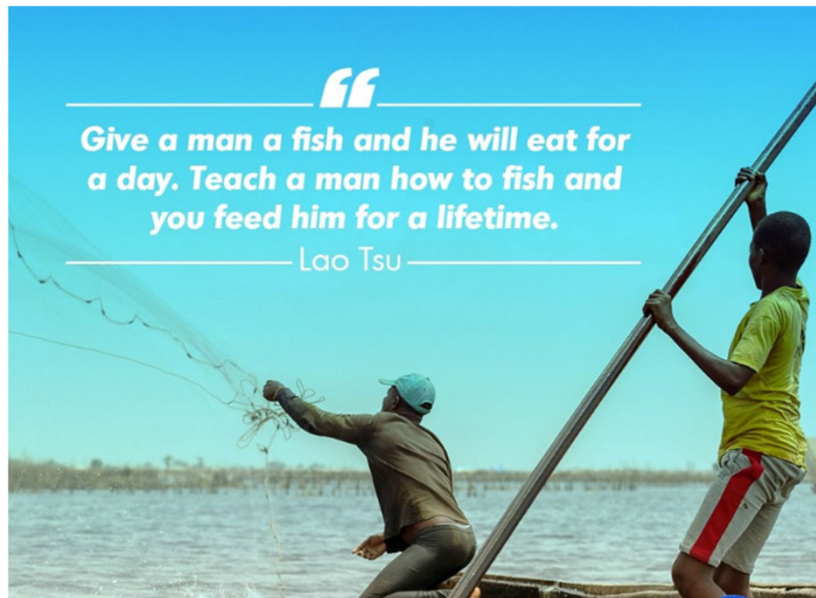
### 3.3 Goals and Rewards

EDUCACIÓN  
SECRETARÍA DE EDUCACIÓN PÚBLICA



#### Remember the Reward Hypothesis

- *"Give a man a fish and he'll eat for a day..."*
- *Teach a man to fish and he'll eat for a lifetime."*



- *"Give a man a taste for fish and he'll figure out how to fish even if the details change..."*



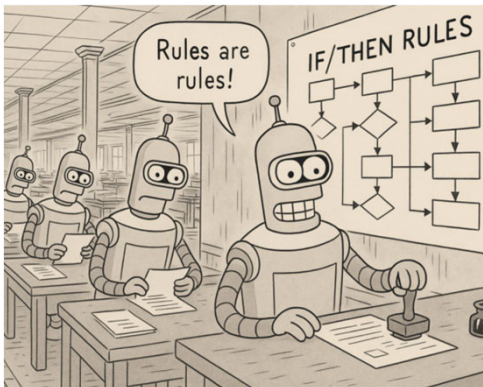
## 3. Markov Decision Process

### 3.3 Goals and Rewards

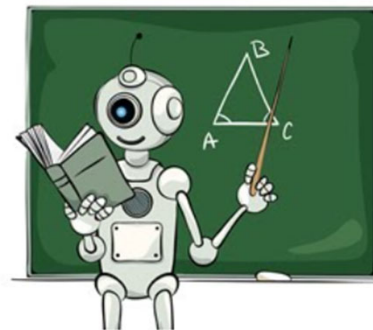


#### Remember the Reward Hypothesis

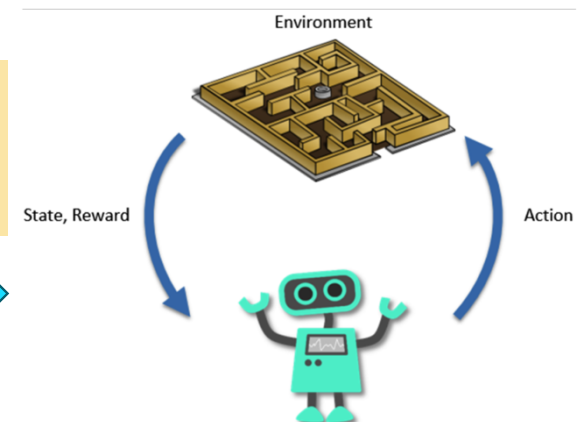
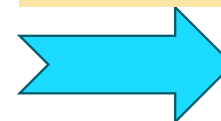
- *"Give a man a fish and he'll eat for a day..."* (GOFAL: Good Old-Fashioned AI).
- *"Teach a man to fish and he'll eat for a lifetime..."* (SL: Supervised Learning).



New  
problems  
provide  
evolution



New  
problems  
provide  
evolution



- *"Give a man a taste for fish and he'll figure out how to fish even if the details change..."* (RL: Reinforcement Learning).

## 3. Markov Decision Process

### 3.4 Returns and Episodes

Consider a robot learning to walk. The reward could be **proportional to the forward motion**. Lurching forward would clearly **maximize immediate reward**. However, this action cause the robot to fall over.



reward  $\propto$  forward motion



reward  $\propto$  forward motion

maximizing the immediate reward





## 3. Markov Decision Process

### 3.4 Returns and Episodes

If the robot maximize total forward motion instead, it would walk quickly but carefully.



reward  $\propto$  forward motion



reward  $\propto$  forward motion



## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Formal definition: Goal of an Agent

The return at time step  $t$ , is simply the sum of rewards obtained after time step  $t$ . We denote the return with the letter  $G$ . The return is a random variable because the dynamics of the MDP can be stochastic.

random variable

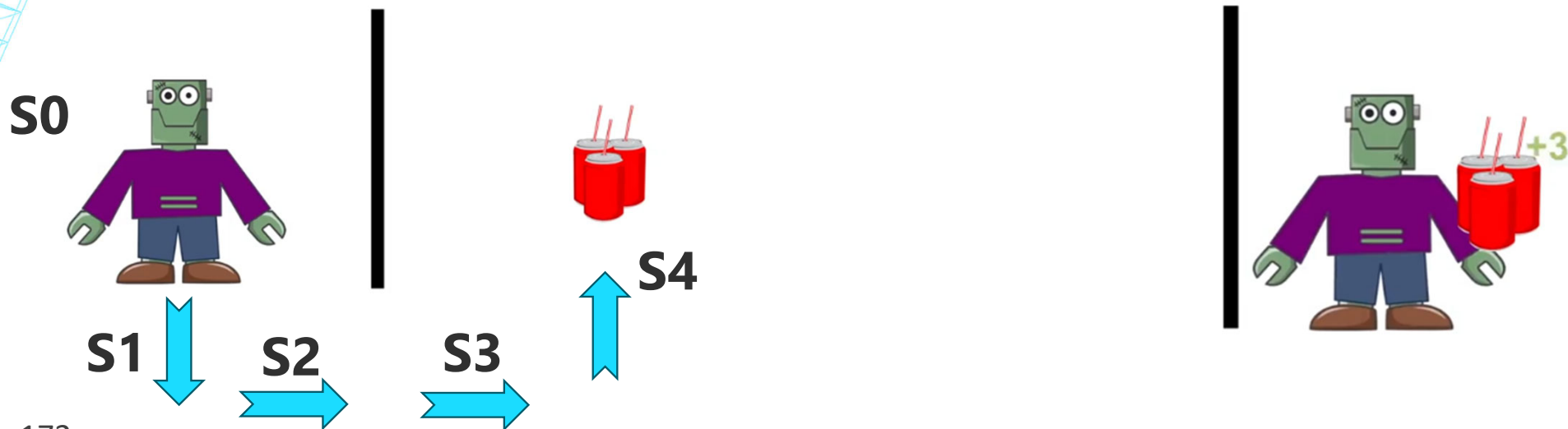


$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

### 3. Markov Decision Process

#### 3.4 Returns and Episodes

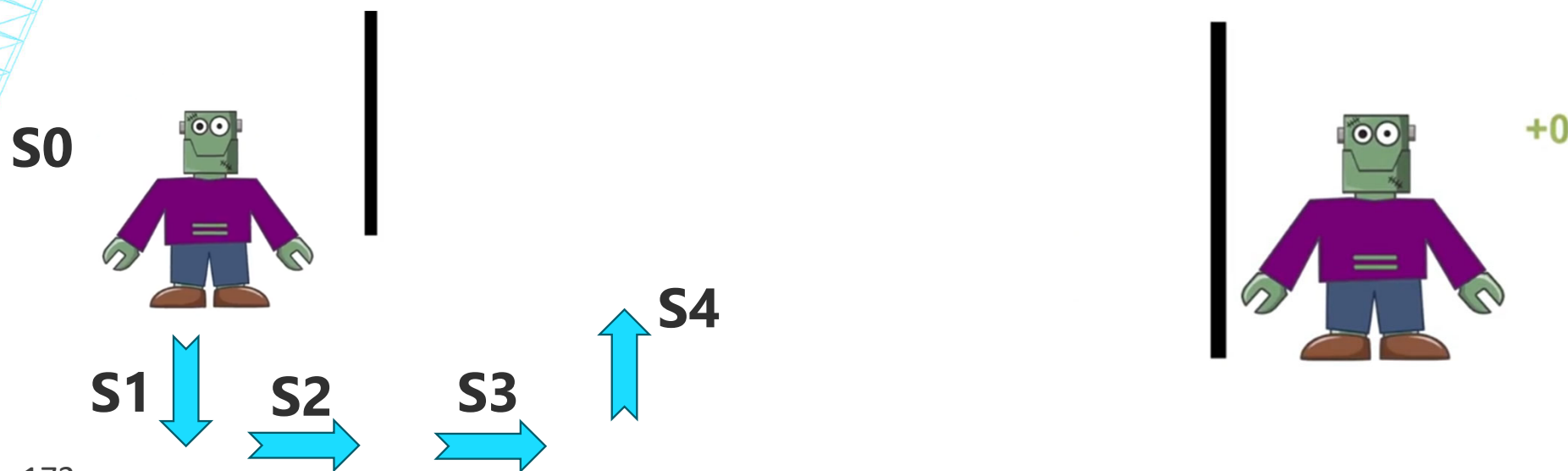
Imagine a can collecting robot starts in **S0**. From this state, the robot always takes the same sequence of actions. Sometimes it might get a large return and sometimes it might get a smaller return. This is due to the randomness in both the individual rewards and the state transitions.



## 3. Markov Decision Process

### 3.4 Returns and Episodes

In general, many different trajectories from the same state are possible.



## 3. Markov Decision Process

### 3.4 Returns and Episodes

This is why we maximize the expected return. For this to be well-defined, the sum of rewards must be finite.

random variable



$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

$$\mathbb{E}[G_t] = \mathbb{E}[ R_{t+1} + R_{t+2} + R_{t+3} + \dots ]$$

## 3. Markov Decision Process

### 3.4 Returns and Episodes

Specifically, let say there is a final time step called capital  $T$  where the agent environment interaction ends. What happens when the interaction ends?



$$\mathbb{E}[G_t] = \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T]$$

final time step

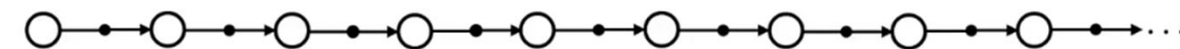


## 3. Markov Decision Process

### 3.4 Returns and Episodes

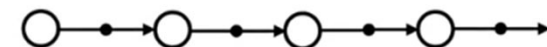
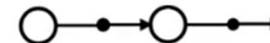
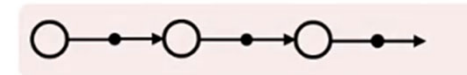
In the simplest case, the interaction naturally breaks into chunks called **episodes**. Each episode begins independently of how the previous one ended. At termination, the agent is reset to a start state. Every episode has a final state which we call the **terminal state**. We call these tasks **episodic tasks**.

#### Episodic Tasks



#### Episodic Tasks

episode



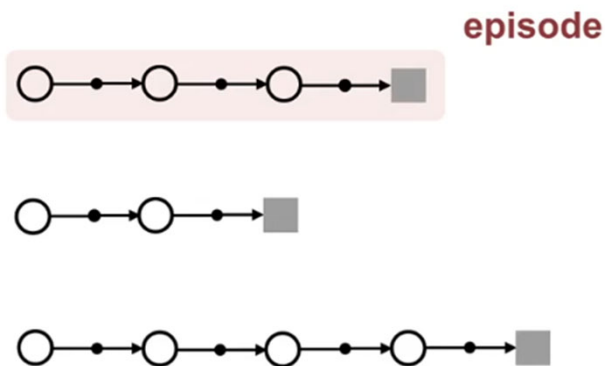
## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Example

Consider the game of chess. A game of chess always ends with a checkmate, draw, or resignation. What would an episode look like when playing chess?

#### Episodic Tasks



#### Example : Episodic Task



Every game of chess  
has a final move

#### Example : Episodes



Each game of  
chess is an episode



## 3. Markov Decision Process

### 3.4 Returns and Episodes



#### Differences between Continuing and Episodic Tasks

##### Episodic Tasks

- Interaction breaks naturally into **episodes**
- Each episode ends in a **terminal state**
- Episodes are **independent**

##### Continuing Tasks

- Interaction goes on **continually**
- No **terminal state**

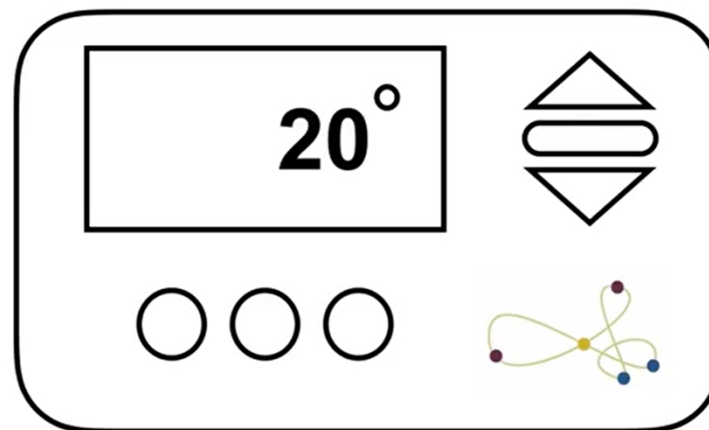
$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Continuing Task Example

Consider a smart thermostat which regulates the temperature of a building. This can be formulated as a continuing task since the thermostat never stops interacting with the environment.

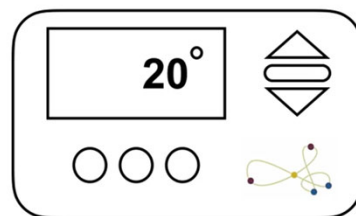
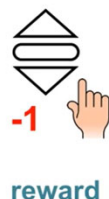


## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept – Continuing Task Example

The state could be the current temperature along with details of the situation like the time of day and the number of people in the building. There are just two actions, turn on the heater or turn it off. The reward to be -1 every time someone has to manually adjust the temperature and zero otherwise. To avoid negative reward, the thermostat would learn to anticipate the user's preferences. So **how can we formulate the return for continuing tasks?** We can try to sum up all the future rewards as we did for episodic tasks. But now, we're summing over an infinite sequence.



naturally formulated as a continuing task

#### Continuing Tasks

- Interaction goes on **continually**
- No **terminal state**

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

$$= \infty?$$



## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept – Continuing Task Example

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k} + \dots$$

How to make sure  $G_t$  is finite?

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

One solution is to discount future rewards by a factor Gamma called the **discount rate**. Gamma is at least zero, but less than one. The return formulation can then be modified to include discounting.

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_{t+k} + \dots$$

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k} + \dots$$

How to make sure  $G_t$  is finite?

**Discount** the rewards in the future by  $\gamma$   
Where  $0 \leq \gamma < 1$

How to make sure  $G_t$  is finite?

**Discount** the rewards in the future by  $\gamma$   
Where  $0 \leq \gamma < 1$

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

The effect of discounting on the return is simple, **immediate rewards contribute more** to the sum. **Rewards far into the future contribute less** because they are multiplied by Gamma raised to successively larger powers of k. Intuitively, this choice makes sense. A dollar today is worth more to you than a dollar in a year.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k} + \dots$$

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

We can concisely write this sum as this expression, which is guaranteed to be finite. The factor gamma is called the **discount rate**.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k} + \dots$$

$$= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Finite as long as  $0 \leq \gamma < 1$

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

Assume  $R_{max}$  is the maximum reward our agent can receive at any time step. We can now upper bound the return  $G_t$  by replacing every reward with  $R_{max}$ . Since  $R_{max}$  is just a constant we can pull it out of the summation. Note that the second factor is just a geometric series.

Assume  $R_{max}$  is the maximum reward the agent can receive

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \leq \sum_{k=0}^{\infty} \gamma^k R_{max} = R_{max} \sum_{k=0}^{\infty} \gamma^k$$



## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

The geometric series evaluates to one divided by one minus Gamma,  $R_{max}$  times one divided by one minus Gamma is finite and is an upper bound on  $G_t$ .

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \leq \sum_{k=0}^{\infty} \gamma^k R_{max} = R_{max} \sum_{k=0}^{\infty} \gamma^k$$

Converging geometric series when  $\gamma < 1$

$$= R_{max} \times \frac{1}{1 - \gamma}$$

<https://www.youtube.com/watch?v=G51Qc8XKtqM>

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

So we know  $G_t$  is finite.

Assume  $R_{max}$  is the maximum reward the agent can receive

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \leq \sum_{k=0}^{\infty} \gamma^k R_{max} = R_{max} \sum_{k=0}^{\infty} \gamma^k$$

Finite!  $\Rightarrow G_t$  is finite

$$= R_{max} \times \frac{1}{1 - \gamma}$$

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

Let's look at the effect of the discount factor on the behavior of the agent. When Gamma equals zero and when Gamma approaches one. When Gamma equals zero the return is just the reward at the next time step. So the agent is **shortsighted** and only cares about immediate expected reward.

#### Effect of $\gamma$ on agent behavior

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k} + \dots$$

$$\gamma = 0$$

$$= R_{t+1} + 0R_{t+2} + 0^2 R_{t+3} + \dots + 0^{k-1} R_{t+k} + \dots$$

$$= R_{t+1}$$

Agent only cares about the immediate reward!

$\Rightarrow$  Short-sighted agent!

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

On the other hand, when Gamma approaches one, the immediate and future rewards are weighted nearly equally in the return. The agent in this case is **more farsighted**.

$$\gamma \rightarrow 1$$

Agent takes future rewards into account more strongly

$\Rightarrow$  Far-sighted agent!

## 3. Markov Decision Process

### 3.4 Returns and Episodes

#### Discounting concept

Let's discuss a simple but important property of the return. It can be written **recursively**. Let's factor out Gamma starting from the second term in our sum. Now the sequence in parentheses is the return on the next time step. So we can just replace it with  $G_{t+1}$ . Now, we have a recursive equation with  $G_t$  on the left and  $G_{t+1}$  on the right. This simple equation is more powerful than it seems.

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) \end{aligned}$$

$$G_t = R_{t+1} + \gamma G_{t+1}$$

This is just  $G_{t+1}$